

Docker Auditor

CAHIER DES CHARGES

Spécifications fonctionnelles et techniques pour le déploiement d'une solution d'audit de sécurité des environnements Docker — Détection CVE/CWE, scoring CVSS v3 et monitoring continu.

PROJET	VERSION	STATUT
Audit & Sécurité Docker	1.0.0 — Mai 2026	Confidentiel

<https://novisec.fr/auditor>

1. Contexte et Objectifs

Dans un contexte où les infrastructures conteneurisées sont devenues la norme, la surface d'attaque s'est considérablement élargie. Les conteneurs embarquent souvent des dépendances obsolètes ou vulnérables (CVE — *Common Vulnerabilities and Exposures*) qui peuvent compromettre l'ensemble du système d'information.

Le projet **Novisec Docker Auditor** vise à fournir une solution clé en main, non intrusive et hautement automatisée pour la détection de vulnérabilités au sein d'environnements Docker.

OBJECTIF PRINCIPAL

Déployer un agent passif (lecture seule) capable de cartographier et d'analyser l'intégralité d'un réseau Docker local pour remonter les failles de sécurité vers un panneau de contrôle (Dashboard) centralisé.

2. Description Générale de la Solution

La solution se divise en deux composants architecturaux majeurs, pensés pour garantir à la fois la sécurité de l'hôte et la facilité de consultation des résultats :

- › **L'Agent d'Audit (Image Docker)** : Un conteneur déployé avec des privilèges stricts de lecture seule. Il s'interface avec le socket Docker de l'hôte de manière sécurisée pour analyser les images et les conteneurs en cours d'exécution sans jamais interférer avec la production.
-
- › **Le Panneau de Contrôle Web (Dashboard)** : Une interface centralisée permettant aux administrateurs systèmes et aux équipes SecOps de visualiser l'état de sécurité de leur parc, de lancer des analyses et de consulter des rapports détaillés.

3. Spécifications Fonctionnelles

3.1. L'Agent d'Audit (Scanner)

- › **Principe de moindre privilège** : L'image s'exécute en mode "Read-Only". Elle ne peut modifier, altérer ou arrêter les autres conteneurs.
- › **Périmètre d'analyse** : Détection automatique de tous les conteneurs du réseau Docker cible et de leurs images sous-jacentes.
- › **Moteur de détection** : Identification des paquets OS (Debian, Alpine, Ubuntu...) et des dépendances applicatives (npm, pip, maven...) pour les croiser avec la base de données NVD (National Vulnerability Database).
- › **Transmission chiffrée** : Envoi des résultats des scans au Dashboard via une API REST sécurisée (TLS 1.3).

3.2. Panneau de Contrôle Web (Dashboard)

Le dashboard est le centre névralgique de la solution. Ses fonctionnalités incluent :

- › **Vue d'ensemble (Overview)** : Graphiques synthétiques du nombre de conteneurs sains vs vulnérables, score de risque global de l'infrastructure.
- › **Matrice de Criticité** : Tri et filtrage des CVE par score CVSS v3 (**Critique / Haut** , **Moyen** , **Faible**).
- › **Fiche Détaillée par Conteneur** : Informations sur le conteneur (ID, Nom, Image, Statut) et liste exhaustive des failles détectées avec des recommandations de remédiation (ex : *"Mettre à jour la librairie log4j vers la version 2.17.1"*).

3.3. Gestion et Programmation des Analyses

L'outil doit offrir une flexibilité totale quant au déclenchement des audits de sécurité pour s'adapter aux processus DevSecOps :

MODE DE DÉCLENCHEMENT	DESCRIPTION	CAS D'USAGE TYPIQUE
Analyse Manuelle Globale	Scan immédiat de l'ensemble du parc Docker connecté à l'agent.	Audit ponctuel de conformité ou réponse à incident.
Analyse Manuelle Ciblée	Scan d'un ou plusieurs conteneurs spécifiquement sélectionnés depuis le dashboard.	Vérification post-déploiement d'un nouveau micro-service.
Analyse Automatique (CRON)	Programmation récurrente (ex : tous les jours à 03h00, ou hebdomadaire).	Supervision continue et détection des failles "Zero-Day" nouvellement publiées.

4. Spécifications Techniques et Sécurité

4.1. Architecture Technologique (Recommandée)

- › **Agent Scanner** : Développé en Go (pour la légèreté et les performances) ou en Python, intégrant un moteur de type Trivy / Clair. Déployé via un fichier `docker-compose.yml` standardisé.
- › **Backend Dashboard** : API développée en Node.js (NestJS) ou Python (FastAPI).
- › **Frontend Dashboard** : Framework moderne de type Vue.js ou React.js. Design system orienté "Dark Mode" pour un aspect purement cybersécurité, reprenant le logo blanc de Novisec.
- › **Base de données** : PostgreSQL pour le stockage des résultats de scan et de l'historique de remédiation.

4.2. Exigences de Sécurité

⚠ SÉCURISATION DU SOCKET DOCKER

L'agent nécessitera un accès au socket (`/var/run/docker.sock`). Il est impératif que ce montage soit réalisé avec le flag de montage en lecture seule (`:ro`) dans la configuration de déploiement pour interdire toute interaction de création, d'altération ou de suppression de conteneurs par l'agent.

- › **Authentification Dashboard** : Accès protégé par authentification forte (MFA fortement recommandé) et gestion par rôles (RBAC : Admin, Viewer).
- › **Mises à jour de la base CVE** : Le scanner doit mettre à jour sa base de données de vulnérabilités avant chaque scan, depuis un flux réseau sortant sécurisé.

5. Planning Prévisionnel

Le développement et la mise en production s'articuleront autour de 4 phases distinctes :

PHASE	LIVRABLES	DURÉE ESTIMÉE
Phase 1 — Architecture & Maquettage	Validation de l'architecture, maquettes (UI/UX) du Dashboard.	2 semaines
Phase 2 — Développement Agent Docker	Agent "Read-Only" opérationnel, remontée des premiers scans.	3 semaines
Phase 3 — Développement Dashboard	API Backend, Interface Web complète, gestion des alertes.	4 semaines
Phase 4 — Intégration & Test (Recette)	Tests d'intrusion sur la solution, vérification de la charge réseau.	2 semaines